

How to automate result reporting

September 21, 2026

<https://github.com/jimb0w/automation>

Jedidiah Morton

Jedidiah.Morton@monash.edu

Research Fellow

Baker Heart and Diabetes Institute, Melbourne, Australia

Monash University, Melbourne, Australia

Contents

1	Introduction	2
2	Data cleaning	3
3	Analysis	8
	References	12

1 Introduction

The premise of this seminar is that you should automate everything you can. Not only are there huge efficiency advantages, it's also the only way to ensure your work is truly reproducible.

This document and the associated Stata do file will hopefully provide some familiarity with the tools to allow you to automate most of the work that you do.

We're using two things that you need in addition to Stata: the texdoc package, and a LaTeX compiler.

2 Data cleaning

Document everything! Say what you're going to do, do it, then comment on it. My latest example is in this reference[?] or at [this link](#).

For example: I'm going to create a random health dataset with the following variables:

- Sex (0 = female; 1 = male)
- Age (in years)
- Diabetes status (0 = no diabetes; 1 = diabetes)
- Smoking status (0 = non-smoker; 1 = smoker)
- LDL-C (in mmol/L)
- Outcome (in outcome units)

As so:

```
cd /home/jimb0w/Documents/PRES/auto/  
clear  
set obs 20000  
set seed 1312  
gen sex = runiformint(0,1)  
gen age = rnormal(60,15)  
gen diabetes = runiformint(0,1)  
gen smoking = runiformint(0,1)  
gen ldl = rnormal(3,0.5)  
gen outcome = rnormal(200,30)+diabetes*rnormal(100,20)  
export delimited using randomhealthdata.csv, replace  
save randomhealthdata, replace
```

Now I'm going to check the variables make sense:

```
. su(sex), detail
```

sex					
	Percentiles	Smallest			
1%	0	0			
5%	0	0			
10%	0	0	Obs	20,000	
25%	0	0	Sum of wgt.	20,000	
50%	0		Mean	.4989	
		Largest	Std. dev.	.5000113	
75%	1	1			
90%	1	1	Variance	.2500113	
95%	1	1	Skewness	.0044	
99%	1	1	Kurtosis	1.000019	

```
. su(diab), detail
```

diabetes					
	Percentiles	Smallest			
1%	0	0			
5%	0	0			
10%	0	0	Obs	20,000	
25%	0	0	Sum of wgt.	20,000	
50%	0		Mean	.494	
		Largest	Std. dev.	.4999765	
75%	1	1			
90%	1	1	Variance	.2499765	

```

95%          1          1      Skewness      .0240017
99%          1          1      Kurtosis      1.000576
. su(smoking), detail

               smoking
-----
Percentiles      Smallest
1%              0          0
5%              0          0
10%             0          0      Obs          20,000
25%             0          0      Sum of wgt.    20,000
50%             1          1      Mean          .50085
               Largest      Std. dev.    .5000118
75%             1          1
90%             1          1      Variance      .2500118
95%             1          1      Skewness      -.0034
99%             1          1      Kurtosis      1.000012
. su(age), detail

               age
-----
Percentiles      Smallest
1%      24.66596      .835979
5%      35.15182      1.354185
10%     40.73132      4.960024      Obs          20,000
25%     49.76038      5.434331      Sum of wgt.    20,000
50%     59.77124          1      Mean          59.82388
               Largest      Std. dev.    14.99574
75%     69.96964      109.9906
90%     78.95889      111.0537      Variance      224.8723
95%     84.55445      111.059      Skewness      -.0096071
99%     94.6009      115.5846      Kurtosis      3.032043
. su(ldl), detail

               ldl
-----
Percentiles      Smallest
1%      1.81618      1.169115
5%      2.166875      1.17411
10%     2.354322      1.184871      Obs          20,000
25%     2.66082      1.20128      Sum of wgt.    20,000
50%     3.000187          1      Mean          2.997539
               Largest      Std. dev.    .50188
75%     3.334033      4.78299
90%     3.636441      4.849288      Variance      .2518836
95%     3.817179      4.876576      Skewness      -.0133279
99%     4.168352      5.034541      Kurtosis      3.031805
. su(outcome), detail

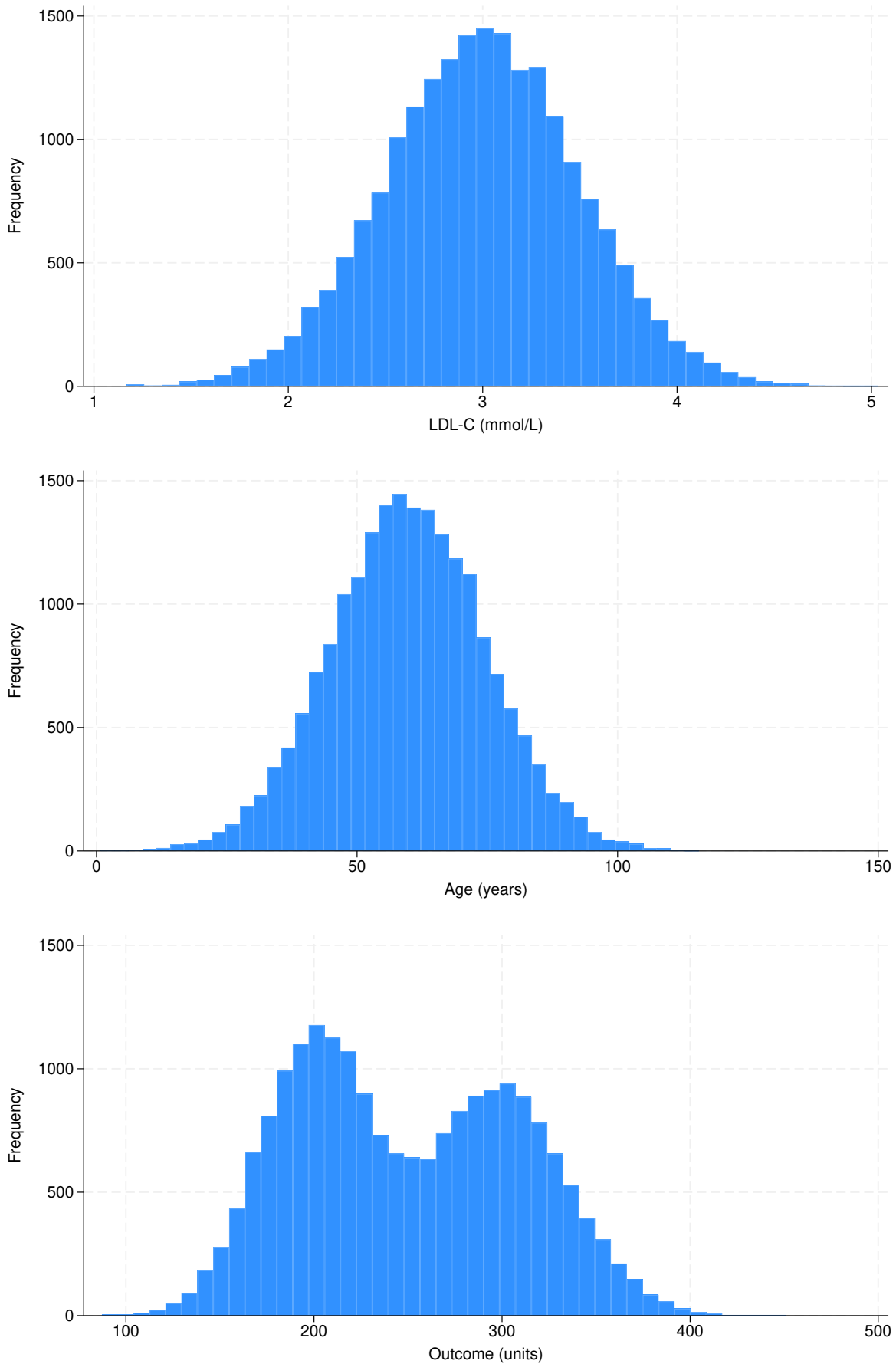
               outcome
-----
Percentiles      Smallest
1%      139.2732      87.21138
5%      162.0865      91.83277
10%     174.7943      91.93818      Obs          20,000
25%     199.8294      92.32668      Sum of wgt.    20,000
50%     244.0778          1      Mean          249.3994
               Largest      Std. dev.    59.80981
75%     299.1514      426.0753
90%     329.6298      430.0988      Variance      3577.214
95%     345.9959      434.5064      Skewness      .163877
99%     374.6187      450.9283      Kurtosis      2.073516

```

It looks like these variables make sense and there don't appear to be any errors with the data. But we should check further: let's produce a histogram to check the continuous variables. The code below produces Figure 2.1.

```
mkdir GPH
hist ldl, frequency ///
xtitle("LDL-C (mmol/L)")
graph save GPH/LDL_hist, replace
hist age, frequency ///
xtitle("Age (years)")
graph save GPH/Age_hist, replace
hist outcome, frequency ///
xtitle("Outcome (units)")
graph save GPH/Outcome_hist, replace
graph combine ///
GPH/LDL_hist.gph ///
GPH/Age_hist.gph ///
GPH/Outcome_hist.gph ///
, cols(1) altshrink xsize(3)
graph export GPH/distcheck.pdf, as(pdf) replace
```

Figure 2.1: Histograms of LDL-C, Age, and outcome.



It appears that our age and LDL-C variables are normally distributed and seem reasonable. there is an odd, bimodal distribution in the outcome variable.

In terms of automation, the key thing here was producing my Figures using code, rather than a graphics program, and automatically exporting them to pdf, before importing them and displaying them within the document. Notice also that Figure 2.1 hyperlinks to the figure if referenced properly, so if you update the figure order later, you don't have to re-number all your figure references in the text. That means if you change anything in your analysis, add a figure, etc., you don't need to re-make the figure, just re-run the code and it will re-appear updated in this pdf file.

3 Analysis

Before beginning our analysis, we should present a summary table of population characteristics. We are most interested in diabetes, so we intend to stratify this table by diabetes type.

Here, we are going to collect the results from the Stata output and store them in a matrix, before saving that matrix to the Stata dataset editor and formatting the outputs for presentation in a Table. It helps to first think about the Table you want to present before collecting the results. The Table 3.1 is what we're aiming for.

Table 3.1: Table we are aiming for.

Overall		Diabetes status	
		No diabetes	Diabetes
N	N	N	N
Females	N (%)	N (%)	N (%)
Smoking status	N (%)	N (%)	N (%)
Age	median (IQR)	median (IQR)	median (IQR)
LDL-C (mmol/L)	median (IQR)	median (IQR)	median (IQR)
Outcome	median (IQR)	median (IQR)	median (IQR)

Data are presented as N, N (% of column total), or median (IQR).

Now that we know what we are aiming for, we need to collect the values. Stata's *return list* command is useful here: it tells you what is stored in Stata's memory, and thus how we can access it. For example:

```
. use randomhealthdata, clear
. su(ldl), detail
```

ldl				
	Percentiles	Smallest		
1%	1.81618	1.169115		
5%	2.166875	1.17411		
10%	2.354322	1.184871	Obs	20,000
25%	2.66082	1.20128	Sum of wgt.	20,000
50%	3.000187		Mean	2.997539
		Largest	Std. dev.	.50188
75%	3.334033	4.78299		
90%	3.636441	4.849288	Variance	.2518836
95%	3.817179	4.876576	Skewness	-.0133279
99%	4.168352	5.034541	Kurtosis	3.031805

```
. return list
scalars:
      r(N) = 20000
    r(sum_w) = 20000
      r(mean) = 2.99753932904005
      r(Var) = .2518835571674297
      r(sd) = .5018800226821443
    r(skewness) = -.0133279276969729
    r(kurtosis) = 3.031805267881794
      r(sum) = 59950.78658080101
      r(min) = 1.169115424156189
      r(max) = 5.03454065322876
      r(p1) = 1.816179573535919
      r(p5) = 2.166874527931213
      r(p10) = 2.354321837425232
      r(p25) = 2.660820245742798
```

```

r(p50) = 3.000186562538147
r(p75) = 3.334033250808716
r(p90) = 3.636441469192505
r(p95) = 3.817179083824158
r(p99) = 4.168352127075195

```

We can access any one of those statistics and store it for use in our table. Annoyingly *tabulate* doesn't store anything useful, so you have to use the *matcell* option.

So, let's get all the data we need for our table:

```

count
local a = r(N)
count if sex == 0
local b = r(N)
count if smoking == 1
local c = r(N)
su(age), detail
mat A1 = (r(p50),r(p25),r(p75))
su(1dl), detail
mat A2 = (r(p50),r(p25),r(p75))
su(outcome), detail
mat A3 = (r(p50),r(p25),r(p75))
mat B = (0`a`,... ///
        \0`b`,... ///
        \0`c`,... ///
        \A1\A2\A3)
forval i = 0/1 {
count if diabetes == `i`
local a = r(N)
count if sex == 0 & diabetes == `i`
local b = r(N)
count if smoking == 1 & diabetes == `i`
local c = r(N)
su(age) if diabetes == `i`, detail
mat A1 = (r(p50),r(p25),r(p75))
su(1dl) if diabetes == `i`, detail
mat A2 = (r(p50),r(p25),r(p75))
su(outcome) if diabetes == `i`, detail
mat A3 = (r(p50),r(p25),r(p75))
mat B`i` = (0`a`,... ///
            \0`b`,... ///
            \0`c`,... ///
            \A1\A2\A3)
}
mat C = (B,B0,B1)

. mat l C
C[6,9]

```

	c1	c2	c3	c1	c2	c3	c1	c2
r1	20000	.	.	10120	.	.	9880	.
r2	10022	.	.	5036	.	.	4986	.
r3	10017	.	.	5061	.	.	4956	.
r1	59.771242	49.76038	69.969643	59.866039	49.805601	70.10405	59.671923	49.716722
r1	3.0001866	2.6608202	3.3340333	2.9967493	2.6569303	3.3266593	3.0038455	2.6666859
r1	244.0778	199.82938	299.1514	200.40833	180.11034	220.09869	299.5594	275.5574

```

c3
r1 .
r2 .
r3 .
r1 69.824024
r1 3.3387761
r1 323.75455

```

We have all our results in the matrix, *C*. However, they aren't in a presentable format. If we were to try and make a table out of this, we would have too many decimal places and inconsistent

column numbers. So, we need to convert the variables to string (essentially text, not number) format so that we can manipulate the data into the format we need for our table.

```
mkdir CSV
clear
svmat C
gen D1 = C1[1]
gen D4 = C4[1]
gen D7 = C7[1]
gen E1 = 100*C1/D1
gen E4 = 100*C4/D4
gen E7 = 100*C7/D7
tostring C1 C4 C7, gen(A1 A4 A7) force format(%9.0fc)
tostring _all, replace force format(%9.1f)
gen A = A1 if _n == 1
gen B = A4 if _n == 1
gen C = A7 if _n == 1
replace A = A1 + " (" + E1 + "%)" if inrange(_n,2,3)
replace B = A4 + " (" + E4 + "%)" if inrange(_n,2,3)
replace C = A7 + " (" + E7 + "%)" if inrange(_n,2,3)
replace A = C1 + " (" + C2 + ", " + C3 + ")" if _n >=4
replace B = C4 + " (" + C5 + ", " + C6 + ")" if _n >=4
replace C = C7 + " (" + C8 + ", " + C9 + ")" if _n >=4
keep A B C
gen row = ""
replace row = "N" if _n == 1
replace row = "Females" if _n == 2
replace row = "Smoking status" if _n == 3
replace row = "Age" if _n == 4
replace row = "LDL-C (mmol/L)" if _n == 5
replace row = "Outcome" if _n == 6
order row
export delimited using CSV/table1.csv, delimiter(":") novarnames replace
save table1, replace
```

We have successfully produced our first table. I like to save tables in a CSV subdirectory to make the main folder cleaner. That also makes it easier to bulk save all your results to GitHub.

There are several ways to use tables in a LaTeX document. I use *pgfplotstable* as it allows me to export the file to a csv document and then use LaTeX. You could also simply write a TeX table in Stata and export that.

Table 3.2: Population characteristics.

	Overall	Diabetes status	
		No diabetes	Diabetes
N	20,000	10,120	9,880
Females	10,022 (50.1%)	5,036 (49.8%)	4,986 (50.5%)
Smoking status	10,017 (50.1%)	5,061 (50.0%)	4,956 (50.2%)
Age	59.8 (49.8, 70.0)	59.9 (49.8, 70.1)	59.7 (49.7, 69.8)
LDL-C (mmol/L)	3.0 (2.7, 3.3)	3.0 (2.7, 3.3)	3.0 (2.7, 3.3)
Outcome	244.1 (199.8, 299.2)	200.4 (180.1, 220.1)	299.6 (275.6, 323.8)

We can see from the table that the Outcome seems to be higher in people with diabetes (median (IQR): 299.6 (275.6, 323.8)) than without diabetes (median (IQR): 200.4 (180.1, 220.1)).

Okay, now it's your turn. Use these tools to run a linear regression analysis comparing each explanatory variable with the outcome, show the results in a table and a figure that is completely automatic. Then comment on the results in the text using results generated automatically.

References